

BARYSGUARD

Administrator & Engineer Manual

Complete reference: features, deployment, and operations

v2.0 · 2025

Threat Intelligence & Incident Response Platform

1. Introduction

BarysGuard v2 is a desktop threat intelligence and incident-response platform for Windows. It enables security analysts, administrators, and engineers to detect malware artifacts, scan files and process memory with YARA rules, check file hashes against VirusTotal, collect network intelligence, manage multiple remote hosts, and perform network isolation — all from a single workstation.

1.1 System Requirements

- OS: Windows 10 / 11 (64-bit) on both analyst workstation and monitored hosts
- Python 3.11+ with PyQt6, requests, psutil, cryptography
- Network access for API calls (VirusTotal, AbuseIPDB)
- yara64.exe or yara-python for YARA scanning
- pywinrm for automated remote agent deployment (optional)

1.2 First Launch

Run the application from the project root:

```
python main.py
```

On first launch, config.json is created automatically. Open the Settings tab to enter your API keys (VirusTotal, AbuseIPDB).

2. Feature Reference

2.1 Dashboard

The Dashboard provides a real-time overview of scanning activity across four sub-tabs: Overview (live counters and event feed), Scans (scan statistics), Network (network lookup results), and Hosts (remote host activity). Every significant finding from any tab is automatically logged to the event feed.

2.2 Hash Lookup

Enter one or more SHA-256, MD5, or SHA-1 file hashes to query the VirusTotal API. Results show the detection ratio, file name, file type, and a direct link to the VirusTotal report. The free VT API supports 4 requests/minute — configure the delay in Settings.

2.3 IOC Collection

Collect Indicators of Compromise from the local host: running processes, active network connections, and autorun registry entries. Suspicious processes (outside C:\Windows and C:\Program Files) are highlighted automatically.

2.4 YARA Scanner (local)

Scan a local file or directory using YARA rules. The scanner ships with 40+ built-in rules covering ransomware (LockBit, BlackCat/ALPHV), RATs (AsyncRAT, QuasarRAT, Remcos), credential stealers (RedLine, Vidar), C2 frameworks (Sliver, Havoc, Chisel), and offensive tools (LOLBins, AMSI bypass, Defender tampering). You can also write custom rules in the built-in editor.

- Select rules from the left panel (use the search box to filter 40+ rules).
- Choose a target file or folder using the File / Folder buttons.
- Click Scan YARA. Results appear in the table with severity color coding.

[!] YARA scanning requires yara64.exe in the project folder or yara-python installed.

2.5 Network Intel

Look up IP addresses and domains against AbuseIPDB and VirusTotal. Results include abuse confidence score, country, ISP, and threat categories.

2.6 Report Builder

Generate incident reports in HTML format. Reports aggregate findings from all tabs, include timestamps, severity levels, and recommendations. Enable auto-save in Settings to export a report after every scan automatically.

2.7 Memory Scanner (local)

List all running local processes and scan a selected process executable with YARA rules. Useful for detecting in-memory malware that has not written files to disk. Requires elevated privileges for protected processes.

2.8 Quarantine

Move suspicious files to an isolated quarantine folder. Quarantined files are renamed to prevent accidental execution. You can restore or permanently delete them from this tab.

2.9 Settings

Configure the application:

- API keys for VirusTotal, AbuseIPDB
- Default folders for results and quarantine
- VirusTotal request rate limit
- Auto-save reports toggle
- Interface language: English / Russian / Kazakh

2.10 Hosts — Multi-Host Remote Scanning

The Hosts tab lets you manage and scan multiple Windows machines from a single BarysGuard instance. Each remote host runs a lightweight Flask HTTPS agent (agent.exe) on port 5555. Select a host from the left panel to access its five sub-tabs.

Sub-tab: Status

Shows host details (IP, port, last ping, last scan) and provides quick-action buttons: Ping, Deploy Agent, and navigation shortcuts to other sub-tabs.

Sub-tab: File Scan

Run YARA, IOC collection, and file hash scanning on a remote path. Contains two inner tabs:

- Scanning — compact rule selector with search filter, path input, IOC/hash options, Scan button.
- Custom Rules — full YARA rule editor. Write rules on the right, manage saved rules on the left.

Sub-tab: Memory Scan

List all processes running on the remote host and scan their memory with YARA rules. Contains two inner tabs:

- Scanning — process table (with filter), rule selector with search, Scan Memory / Stop buttons.
- Custom Rules — same full YARA rule editor as in File Scan.

Sub-tab: Results

Displays all scan findings in a table (Rule, Severity, File/Process) with a scan log below. Findings are color-coded by severity: Critical (red), High (orange), Medium (blue), Low (green).

Sub-tab: Isolation

Perform network isolation on the remote host via Windows Firewall. See Section 2.11.

2.11 Network Isolation (Incident Response)

Network isolation blocks all inbound and outbound traffic on a compromised host while preserving the analyst's access. This is a critical incident-response capability that can contain an active attack without requiring physical access to the host.

[!!] Isolation modifies Windows Firewall profile defaults. The agent MUST be running as Administrator, otherwise isolation will fail with a 500 error.

How it works

Isolation uses Set-NetFirewallProfile to set the firewall default action to Block for all profiles (Domain, Private, Public), then adds explicit Allow rules for the management IP. This approach is correct: Allow rules override the profile default, but NOT explicit Block rules — so the management connection is always preserved.

- Enter your management IP (pre-filled with your current IP) in the field.
- Click 'Isolate Host'. Confirm the dialog.
- The host is isolated. You can still reach the agent from your IP.
- Click 'Restore Network' to remove isolation rules and reset the firewall profile.

[!] If isolation was applied but Restore fails (timeout), go to the isolated host physically and run: netsh advfirewall reset

Firewall rules created during isolation

- IOCIsoate_AllowMgmt_In — allows inbound from management IP
- IOCIsoate_AllowMgmt_Out — allows outbound to management IP

Profile default: DefaultInboundAction = Block, DefaultOutboundAction = Block

On restore: both rules are deleted and profile defaults are set to NotConfigured.

3. Deploying a Remote Agent

3.1 What the Agent Does

agent.exe (compiled from agent.py) is a self-contained Flask HTTPS server that runs as a Windows Service or standalone process on each monitored host. All traffic is encrypted with a self-signed TLS certificate generated on first run. Every endpoint requires an API token in the X-API-Token header.

API Endpoints

- GET /ping — returns hostname, OS, agent version
- GET /info — returns CPU / RAM / disk usage and logged-on users
- POST /scan/yara — runs YARA rules against a remote path
- POST /scan/ioc — collects processes, connections, autoruns
- POST /scan/memory — scans process memory with YARA
- POST /scan/hashes — returns SHA-256 hashes of files in a path
- GET /processes — returns the running process list
- GET /network/status — returns isolation status (isolated: true/false)
- POST /network/isolate — applies network isolation (requires admin)
- POST /network/restore — removes isolation rules and resets firewall profile

3.2 Method A — Manual Deployment

Use this method when WinRM is not available on the target host.

Step 1 — Copy agent.exe to the target host

agent.exe is a self-contained executable — no Python required on the target. Copy it to C:\IOCAgent\ on the remote machine (USB, shared folder, or file copy).

```
# From your workstation (replace TARGET-HOST with actual name or IP):
copy agent\dist\agent.exe \\TARGET-HOST\c$\IOCAgent\
```

Step 2 — Run agent.exe to generate the token

```
# On the target host, open PowerShell as Administrator:
cd C:\IOCAgent
.\agent.exe
# Output:
# [IOCAgent] Generating self-signed certificate...
# [IOCAgent] v2.0 starting on https://0.0.0.0:5555
# [IOCAgent] Token: a3f7c9d2... (32-char hex)
```

[OK] The token is also saved in C:\IOCAgent\token.txt — copy it for Step 5.

Step 3 — Install as Windows Service (recommended)

```
# Run as Administrator:
.\agent.exe --install
.\agent.exe --start
# Or: net start IOCAgent
```

[!] Running as a service ensures the agent starts automatically after reboot and has SYSTEM privileges needed for network isolation.

Step 4 — Allow port 5555 in Windows Firewall

```
netsh advfirewall firewall add rule name="IOCAgent" ^
dir=in action=allow protocol=TCP localport=5555
```

Step 5 — Add the host in BarysGuard

Open the Hosts tab, click '+ Add', and fill in:

- Name: any label (e.g. SRV-DC01)
- IP: the target host IP address
- Port: 5555
- Token: the value from token.txt

Step 6 — Test connectivity

Click the Ping button. The status indicator should turn green (online).

3.3 Method B — Automated Deployment via UI

If the target host has WinRM enabled (port 5985), you can deploy the agent directly from the BarysGuard UI without touching the remote machine manually.

Prerequisites on the target host

```
# Run as Administrator on the TARGET host:
winrm quickconfig
# For NTLM (domain environment):
Set-Item WSMan:\localhost\Service\Auth\NTLM -Value true
```

[!] WinRM is disabled by default on Windows workstations.

Build agent.exe (run once on your workstation)

```
cd agent
build.bat      # Produces: agent\dist\agent.exe
```

Deploy from UI

- Open the Hosts tab and select or add a host.
- Go to the Status sub-tab and click 'Deploy Agent'.
- Enter the target host IP, administrator username, and password.
- Click OK — BarysGuard copies agent.exe, installs the service, and retrieves the token automatically.

3.4 Verifying the Agent

```
$tok = Get-Content C:\IOCAgent\token.txt
Invoke-WebRequest -Uri https://TARGET-IP:5555/ping `
-headers @{"X-Api-Token"=$tok} -SkipCertificateCheck
# Expected: {"status": "ok", "hostname": "TARGET-HOST", ...}
```

4. Writing Custom YARA Rules

Both the local YARA Scanner tab and the remote Hosts tab (File Scan and Memory Scan) include a full YARA rule editor. Custom rules are stored in memory for the current session and can be written, edited, updated, and deleted.

4.1 Using the Rule Editor

- Open the 'Custom Rules' inner tab inside File Scan or Memory Scan.
- Write your rule in the editor on the right. The rule name is auto-detected from the 'rule <Name>' declaration.
- Click 'Add / Update Rule'. The rule appears in the list on the left and is checked in the rule selector.
- Click a rule in the left list to load it back into the editor for modification.
- Click 'Delete Rule' to remove a custom rule from both the list and the rule selector.

4.2 YARA Rule Template

```
rule MyRule {
  meta:
    description = "Detects suspicious pattern"
    author      = "Your name"
    severity    = "high"
  strings:
    $s1 = "malicious_string" ascii nocase
    $s2 = { 4D 5A 90 00 }           // hex pattern
    $re = /http[s]?:\V[0-9]+\.[0-9]+/ // regex
  condition:
    any of them
}
```

[!] YARA 4.5+ treats unreferenced strings as errors. Every declared string must appear in the condition.

4.3 Built-in Rules Reference

The following rules are included (40+ total):

- Ransomware: LockBit, BlackCat_ALPHV, Ransomware_Generic
- RATs: AsyncRAT, QuasarRAT, Remcos_RAT
- Stealers: RedLine_Stealer, Vidar_Stealer, AgentTesla
- C2 Frameworks: Sliver_C2, Havoc_C2, Chisel_Tunnel, CobaltStrike_Beacon
- Droppers: GuLoader, Emotet_Loader
- Exploits: Log4Shell_CVE_2021_44228
- Web Shells: WebShell_PHP, WebShell_ASPX
- Defense Evasion: AMSI_Bypass, DefenderTampering, Packed_UPX
- Lateral Movement: Mimikatz_Strings, LOLBins_Abuse
- Persistence: Scheduled_Task_Abuse, Network_Recon

5. Security Notes

- The API token is a 64-character random hex string generated once at installation and stored in token.txt.
- All agent traffic uses TLS (self-signed certificate). The client uses verify=False — acceptable for isolated internal networks.
- Protect token.txt with filesystem ACLs. Anyone with the token can control the agent.
- WinRM credentials used during Deploy are not stored by the application.
- config.json stores API keys in plain text — do not commit to version control.
- The agent runs as SYSTEM when installed as a service. Restrict access to C:\IOCAgent\.
- Network Isolation requires the agent to run as Administrator (SYSTEM service satisfies this).

[!!] Always specify your management IP before isolating a host. If you isolate without a valid management IP, you will lose remote access.

6. Troubleshooting

Agent shows 'offline' after ping

- Verify agent.exe is running: tasklist | findstr agent
- Check port 5555 is open: netstat -ano | findstr 5555
- Confirm the token in hosts.json matches token.txt on the remote host.
- Check Windows Firewall allows port 5555 inbound.

Isolation fails with 500 error

- The agent is not running as Administrator.
- Fix: reinstall the agent as a Windows Service (runs as SYSTEM).

```
.\agent.exe --uninstall
.\agent.exe --install
.\agent.exe --start
```

Restore fails with connection timeout

- Isolation was applied but the management IP was not allowlisted correctly.

Fix: go to the isolated host physically and run:

```
netsh advfirewall reset
```

YARA scan returns COMPILE_ERR

- The rule contains a syntax error. Open the Rule Editor and check the rule text.
- YARA 4.5+ requires every declared string to be used in the condition.

VirusTotal returns 'Quota exceeded'

- Increase the rate limit delay in Settings (default 15 s for free API).
- Verify the API key at virustotal.com.

Deploy fails — pywinrm not installed

```
pip install pywinrm
```

Deploy fails — WinRM connection error

- Enable WinRM on target: winrm quickconfig
- Check firewall rules for port 5985 on the target host.